NPC Steering & Driving AI: Comprehensive Architecture Study Guide, Algorithmic Deep-Dive, and Viva Preparation Handbook

> **Project Research Question:**
> *"How does fuzzy-logic decision making affect Non-Player Character (NPC) behavior compared with alternative AI decision architectures?"*

Table of Contents

1. Special In-Depth Breakdown: "AI Profiler & Frame Graph" Telemetry Panel
2. Part 1: Executive Summary & Project Purpose
3. Part 2: System Architecture & Simulation Lifecycle
4. Part 3: TypeScript for B.Tech CS Students (Bridging from Python, C++, and Java)
5. Part 4: The Five AI Architectures: Deep In-Depth Analysis
    - 4.1 Finite State Machine (FSM) / Simple Decision Logic
    - 4.2 Behavior Trees (BT)
    - 4.3 Fuzzy Logic Controller
    - 4.4 Goal-Oriented Action Planning (GOAP)
    - 4.5 Utility AI (Response Curves & Inertia Hysteresis)
6. Part 5: Direct Comparative Analysis Matrix
7. Part 6: Masterclass on Fuzzy Logic for NPC Steering
8. Part 7: Experimental Methodology, Scenarios & Scientific Fairness
9. Part 8: Performance Metrics, Statistical Analysis & CSV Generation
10. Part 9: End-to-End Execution Flowchart & Code Traces
11. Part 10: Presentation & Viva Defense Preparation (25+ Questions & Answers)
12. Part 11: 7-Day Structured Study Plan & Ultimate Exam Cheat Sheet

---

Special In-Depth Breakdown: "AI Profiler & Frame Graph" Telemetry Panel



This floating telemetry widget is located at the bottom-right corner of the simulation arena canvas. It is implemented in `src/components/PerformanceGraph.tsx` and powered by the telemetry engine in `src/game/PerformanceProfiler.ts`.

What Is This Panel For?

In academic AI research and real-time game development, an AI decision model is evaluated not merely by whether it looks convincing, but by its **computational budget footprint**. Modern game physics and rendering run at 60 FPS (giving an absolute frame budget of **16.66 milliseconds** per frame). If an NPC AI takes 5ms to deliberate, it steals 30% of the CPU budget away from rendering, audio, and physics.

This panel provides real-time, microsecond-level (μs) telemetry showing the exact CPU time consumed by each of our five AI decision models, comparing their algorithmic complexity against an empirical baseline.

Detailed Element-by-Element Breakdown of the Image

Section / UI Element	Exact Value in Image	Underlying Code & Mechanism	Technical Meaning & Viva Significance
Panel Header Title	`AI PROFILER & FRAME GRAPH`	`PerformanceGraph.tsx: L181-L184`	Indicates that the HUD element is tracking both overall browser frame latency and decomposed AI decision execution times.
Benchmark Button	`✦ Benchmark`	`PerformanceProfiler.ts: L208-L260`	Triggers `runMicroBenchmark(engine, 300)`: runs a deterministic 300-cycle batch evaluation on all 5

Section / UI Element	Exact Value in Image	Underlying Code & Mechanism	Technical Meaning & Viva Significance
			AI algorithms in isolation, returning empirical mean, min, max, and variance without rendering overhead.
Window Controls	Minimize icon ⤢	`PerformanceGraph.tsx: L200-L205`	Collapses the expanded graph into a lightweight floating pill showing FPS, Frame Time, and Total AI Load.
FRAME CADENCE	`7.1ms (141 FPS)`	Measured via `performance.now()` in `GameCanvas.tsx: L101-L107`	The time elapsed between two consecutive screen redraws. `7.1ms` indicates the browser is rendering at 141 Hz (a high-refresh gaming monitor). If this exceeds 16.6ms, FPS drops below 60.
AI TICK LOAD	`5 µs (0.01ms)`	Sum of all active AI execution times in the current frame (`PerformanceProfiler.ts: L131-L146`)	The cumulative CPU time required to evaluate all active NPCs on screen during this exact tick. $5\mu s = 0.005ms$.
BUDGET (16.6ms)	`0.0%`	`(totalAITimeMs / 16.66) * 100` (`PerformanceGraph.tsx: L125`)	Quantifies what fraction of the 16.66ms frame window is consumed by AI logic. At 0.0%, the AI takes almost zero rendering budget, proving real-time scalability.
View Mode Toggle	`Frame & Total AI` vs `5-AI Cost Curves`	`PerformanceGraph.tsx: L253-L273`	Toggle between viewing the aggregated system performance (Frame time vs total AI load) and viewing five individual comparative curves simultaneously.
Y-Axis Scale Dropdown	`Scale: 16.6ms (60 FPS)`	`PerformanceGraph.tsx: L278-L287`	Sets the ceiling for the rolling oscilloscope canvas. `16.6ms` draws a reference dashed green baseline representing the 60 FPS budget limit.
Oscilloscope Rolling Graph	Cyan curve vs Purple line	`drawTelemetryGraph()` in `PerformanceGraph.tsx: L410-L490`	**Cyan Curve (Frame Time):** Spikes represent browser composite phases, garbage collection, and canvas rendering. **Purple Line (Total AI Load):** Flat at the bottom, visually

Section / UI Element	Exact Value in Image	Underlying Code & Mechanism	Technical Meaning & Viva Significance
			demonstrating that AI computation is negligible compared to browser drawing operations.
ALGORITHM COMPUTATIONAL COST	Section Header	`PerformanceGraph.tsx: L298-L302`	Ranks all 5 algorithms side-by-side using Exponential Moving Average (EMA, $\alpha = 0.12$) to eliminate momentary OS scheduling jitter.
GOAP Planner	`24.6µs` (Ratio: `8.8×`)	`ALGORITHM_CONFIG.goap` in `PerformanceProfiler.ts: L45-L50`	Formulates plans by executing an A^* forward search over permutations of world-state atoms (`GOAPPlanner.plan()`). Explores action preconditions and post-effects.
Utility AI	`27.9µs` (Ratio: `10.0×`)	`ALGORITHM_CONFIG.utility` in `PerformanceProfiler.ts: L38-L43`	Evaluates 4 continuous mathematical response curves (Exponential, Logistic Sigmoid, Linear, and Quadratic) and computes action inertia hysteresis.
Fuzzy Logic	`60.8µs` (Ratio: `21.7×`)	`ALGORITHM_CONFIG.fuzzy` in `PerformanceProfiler.ts: L32-L37`	**Highest execution time.** Must compute triangular (`trimf`) and trapezoidal (`trapmf`) memberships for distance, health, and obstacle danger, evaluate 5 continuous Mamdani rules, and perform centroid defuzzification via vector blending.
Behavior Tree	`34.4µs` (Ratio: `12.3×`)	`ALGORITHM_CONFIG.behavior_tree` in `PerformanceProfiler.ts: L26-L31`	Traverses a hierarchical composite tree (Selector and Sequence nodes), evaluating conditional predicates and action execution leaves.
Simple FSM	`2.8µs` (Ratio: `1.0×`)	`ALGORITHM_CONFIG.simple` in `PerformanceProfiler.ts: L20-L25`	**Lowest execution time.** Baseline ($1.0\times$). Executes hard-coded `if-else` branching on scalar thresholds. Extremely fast $O(1)$ decision making, but visually rigid.

Section / UI Element	Exact Value in Image	Underlying Code & Mechanism	Technical Meaning & Viva Significance
Cost Insight Box	Bottom Text Callout	`PerformanceGraph.tsx: L368-L378`	Summarizes the academic trade-off: *"GOAP uses A* graph search over actions (~8–10× cost of FSM). Simple FSM executes lowest (~8µs) but exhibits rigid transitions."*

Part 1: Executive Summary & Project Purpose

1.1 The Problem Being Investigated

In video games and autonomous robotics, Non-Player Characters (NPCs) often exhibit rigid, unnatural, or erratic behavior when navigating dynamic combat environments. Traditional Finite State Machines suffer from **state explosion** and **binary boundary snapping** (where an agent violently jitters back and forth across a hard threshold like 30% health). While more modern architectures like Behavior Trees and GOAP solve structural modularity, they still rely on binary Boolean conditions (`isHealthy == true`).

This project scientifically investigates:

> **"How does fuzzy-logic decision making affect NPC behavior compared with alternative AI decision architectures?"**

We compare **Fuzzy Logic** directly against four leading industry and academic paradigms:

1. **Finite State Machines (FSM)** / Simple Decision Logic
2. **Behavior Trees (BT)**
3. **Goal-Oriented Action Planning (GOAP)**
4. **Utility AI (Mathematical Response Curves)**

All five architectures are tested in the exact same 2D vehicle-steering environment, with identical physics, raycast obstacle sensors, combat rules, and opponent behaviors.

1.2 What is an NPC and What is "Steering AI"?

An **NPC (Non-Player Character)** is an autonomous game entity controlled by code rather than a human player.

In our project, an NPC possesses two distinct, layered systems:

1. **Decision Architecture (The "Brain"):** Decides *WHAT* the NPC wants to achieve (e.g., "Chase the player", "Retreat to heal", "Corner-flank around a crate").
2. **Steering Physics (The "Body"):** Computes *HOW* the NPC moves towards that goal using Craig Reynolds' Steering Model:

$$\vec{F}_{\text{steering}} = \text{clampLength}(\vec{V}_{\text{desired}} - \vec{V}_{\text{current}}, F_{\max})$$

$$\vec{V}_{\text{new}} = \vec{V}_{\text{current}} + \vec{F}_{\text{steering}} \cdot \Delta t$$

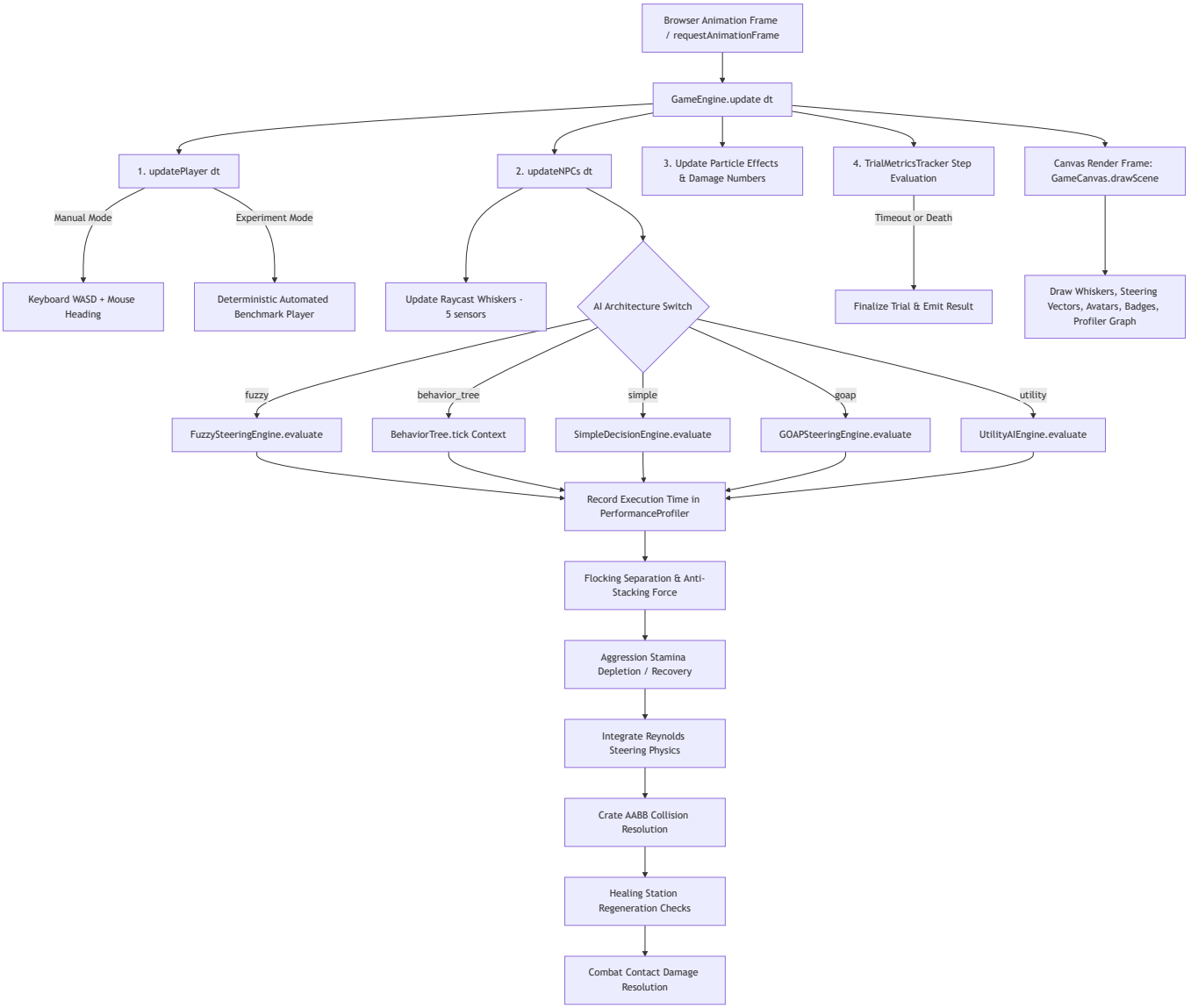
$$\vec{P}_{\text{new}} = \vec{P}_{\text{current}} + \vec{V}_{\text{new}} \cdot \Delta t$$

1.3 The Five Architectures at a Glance

THE 5 NPC DECISION ARCHITECTURES				
Simple FSM	Behavior Trees	Fuzzy Logic	GOAP Planner	Utility AI
Hard if/else states (Chase, Flee, Slide)	Hierarchical Selector/Sequence traversal	Continuous partial truth & Mamdani vector blending	A* graph search over action preconditions	Mathematical curves & scoring
O(1) complexity	Priority-driven	Jitter-free blends	Goal-directed	Inertia damp
Fastest (1.0x)	Modular (12x)	Smooth (22x)	Strategic (9x)	Nuanced (10x)

Part 2: System Architecture & Simulation Lifecycle

2.1 Complete Architectural Flowchart



2.2 The 8-Phase Lifecycle of a Single Simulation Tick

Every single frame in `src/game/GameEngine.ts: L457-L540` executes in this exact strict order:

1. **Delta Time Clamping & Profiler Clock:**
   - `dt` is clamped to $\leq 0.05\text{s}$ (preventing physics "tunneling" if the browser drops frames).
   - `this.profiler.startFrame()` resets per-frame microsecond accumulators.
2. **Player Movement & Automated Opponent Execution:**
   - In sandbox mode: updates player position based on keyboard `WASD` and Spacebar pulse shockwaves.
   - In experiment mode: executes `updateAutomatedPlayer(dt)` using deterministic kiting and automated EMP pulse defense.
3. **NPC Environmental Sensing (Raycasting):**
   - For every NPC, five directional whisker sensors cast rays against all obstacle crates and outer arena walls (`updateSensors(npc)`).
4. **AI Decision-Making Execution:**
   - The engine checks `npc.aiType` and invokes the matching static evaluator.
   - High-precision timestamp `performance.now()` measures the exact microseconds consumed, passing it to `this.profiler.recordAIEvaluation()`.
5. **Flocking & Stamina Dynamics:**
   - Proactive flocking separation applies inverse-square repulsion between nearby NPCs to prevent "conga lines" or clumping.

    - Stamina depletes over ~4 seconds during close-range aggressive pursuit, forcing tired NPCs to slow down to 55% speed.
6. **Reynolds Steering Physics Integration:**
    - $\vec{V}_{\text{new}} = \vec{V}_{\text{old}} + \vec{F}_{\text{steering}} \cdot \Delta t$. Velocity is clamped to `currentMaxSpeed`.
    - Box collision with crates is resolved using Axis-Aligned Bounding Box (AABB) clamping.
7. **Healing Shrine & Combat Damage Checks:**
    - If an NPC is inside a Healing Shrine, health regenerates at $25–30\ \text{HP}/\text{s}$.
    - If an NPC's collision box touches the player ($d \leq 25\text{px}$), the player takes $22\ \text{damage}/\text{second}$.
8. **Trial Metrics Evaluation & Termination:**
    - If in experiment mode, `TrialMetricsTracker.onStep()` accumulates distance, debounced attacks, and retreat transitions.
    - If time expires or an entity dies, the trial finalizes and emits a `TrialResult`.

Part 3: TypeScript for B.Tech CS Students (Bridging from Python, C++, and Java)

TypeScript is a **statically typed superset of JavaScript**. For a student coming from Java or C++, TypeScript feels like Java with Python's clean syntax.

3.1 Variables: `let` vs `const`

- `const` creates an immutable reference (like `final` in Java or `const` in C++).
- `let` creates a reassignable block-scoped variable.

```
// From src/game/GameEngine.ts: L757-L767
const distToPlayer = Math2D.dist({ x: npc.x, y: npc.y }, playerPos); // Cannot reassign distToPlayer
let moveX = 0; // Can be reassigned
if (this.keysDown.has('w')) moveX -= 1;
```

3.2 Interfaces and Type Aliases

In C++, an `interface` is an abstract class with pure virtual methods. In Java, it is an `interface`. In TypeScript, an `interface` defines the exact shape of an object. The TypeScript compiler enforces that objects match this shape at compile-time, but interfaces generate **zero JavaScript code at runtime**.

```
// From src/types.ts: L1-L10
export interface Vector2 {
  x: number;
  y: number;
}

export type AIType = 'fuzzy' | 'behavior_tree' | 'simple' | 'goap' | 'utility';
```

Java/C++ Equivalent: `AIType` is an `enum`, and `Vector2` is a `class` or `struct` with two `float` fields.

3.3 Union Types and Nullable Types (`| null`, `?`)

In Java, any object can accidentally be `null`, causing `NullPointerException`. TypeScript requires you to explicitly declare if a variable can be null.

```
// 'nearestHealStation' can either be a HealingStation object OR null.
nearestHealStation: HealingStation | null;

// The '?' means the property is optional (can be undefined).
aggressionStamina?: number;
```

3.4 Arrow Functions and Callbacks

In Python, you have `lambda x: x * 2`. In Java 8+, you have `(x) -> x * 2`. In TypeScript, arrow functions `(args) => returnVal` are standard:

```
// From src/game/TrialMetricsCollector.ts: L196-L197
const survivals = trials.map((t) => t.survivalTime);
const npcWins = trials.filter((t) => t.npcWon).length;
```

Python Equivalent:

```
survivals = [t.survivalTime for t in trials]
npc_wins = len([t for t in trials if t.npcWon])
```

3.5 The `this` Keyword & Object Mutations

In Java/C++, `this` always refers to the current class instance. In TypeScript classes, `this` works identically when using standard class methods:

```
// From src/game/GameEngine.ts: L99-L104
public loadPreset(preset: CratePreset) {
  this.currentPreset = preset;
  const layout = getMapLayout(preset, this.width, this.height);
  this.crates = layout.crates;
  this.healStations = layout.healStations;
}
```

Part 4: The Five AI Architectures: Deep In-Depth Analysis

4.1 Finite State Machine (FSM) / Simple Decision Logic

A. Core Idea & Analogy

- **Core Idea:** The agent can only ever exist in one discrete state at any single point in time. Transitions between states occur when specific scalar conditions (like `health <= 30%`) evaluate to `true`.
- **Everyday Analogy:** A traffic light. It is either RED, YELLOW, or GREEN. It cannot be "partially red and partially green."

B. Implementation Details

- **Source File:** `src/ai/simpleLogic.ts`
- **Primary Method:** `SimpleDecisionEngine.evaluate(npc, player, crates, healStations, nearestHealStation)`
- **States Implemented:**
  1. `DIRECT_CHASE`: Straight-line pursuit toward player coordinates.
  2. `FLEE_TO_HEAL`: High-speed retreat towards the nearest healing shrine center.
  3. `EXHAUSTED_STANDOFF`: Backing off and keeping distance while stamina recovers.
  4. `OBSTACLE_SLIDE`: Emergency deflection when the center whisker detects a barrier.

```
// src/ai/simpleLogic.ts: L32-L58
if (healthPercent <= 30 || (npc.isHealing && healthPercent < 75)) {
  currentState = 'FLEE_TO_HEAL';
  badge = 'SIMPLE: Flee (Low HP)';
  targetPoint = { x: nearestHealStation.x + nearestHealStation.width / 2, ... };
} else if (npc.isExhausted) {
  currentState = 'EXHAUSTED_STANDOFF';
  badge = `SIMPLE: Tired (${Math.ceil(npc.aggressionStamina ?? 0)}s)`;
  const awayDir = Math2D.normalize(Math2D.sub(npcPos, playerPos));
  targetPoint = Math2D.add(npcPos, Math2D.scale(awayDir, 140));
} else {
  currentState = 'DIRECT_CHASE';
  targetPoint = playerPos;
}
```

C. Decision Inputs Table

Input Name	Origin	Transformation in Code	Effect on Decision
`npc.health`, `npc.maxHealth`	Simulation entity	`(health / maxHealth) * 100`	Hard threshold: $\leq 30\%$ forces `FLEE_TO_HEAL`.
`npc.isHealing`	Shrine contact	Boolean flag	Latches fleeing state until health reaches 75%.
`npc.isExhausted`	Stamina system	Boolean flag	Triggers `EXHAUSTED_STANDOFF`.

Input Name	Origin	Transformation in Code	Effect on Decision
`centerSensor`	0° forward whisker	`hitDistance < 50`	Triggers `OBSTACLE_SLIDE` via wall tangent projection.

D. Worked Numerical Example

- **Inputs:** `health = 25%`, `distance = 100px`, `centerSensor.isBlocked = false`.
- **Step 1:** Rule 1 checks: `healthPercent (25) <= 30` $\rightarrow$ **TRUE**.
- **Step 2:** State becomes `FLEE_TO_HEAL`. Target point set to Shrine center.
- **Step 3:** Speed multiplier set to $1.15\times$ ($175 \times 1.15 = 201.25$ px/s).
- **Step 4:** Steering force generated directly towards shrine.

E. Professor Viva Questions & Answers

1. **Q: Why does the FSM exhibit jitter when health is right at 30%?**
   A: Without hysteresis, taking damage to 30% triggers retreat, but recovering 1 HP to 31% instantly switches it back to attack, creating violent rapid-fire toggling. We solved this in code by adding a state latch: once healing, it stays in `FLEE_TO_HEAL` until reaching 75% HP.
2. **Q: How does FSM obstacle avoidance differ from the other architectures?**
   A: FSM uses a naive single-feeler tangent slide (`simpleLogic.ts: L64-L91`). It only checks the center forward sensor and projects velocity onto the wall normal's perpendicular vector.

4.2 Behavior Trees (BT)

A. Core Idea & Analogy

- **Core Idea:** Hierarchical directed acyclic graph composed of composite nodes (Selectors and Sequences), decorators, and leaves (Conditions and Actions). Execution returns `SUCCESS`, `FAILURE`, or `RUNNING`.
- **Everyday Analogy:** An emergency hospital triage checklist. Check Life Threatening Condition $\rightarrow$ if YES, treat immediately; if NO, proceed down the checklist to Urgent Care, then General Consult.

B. Implementation Details

- **Source File:** `src/ai/behaviorTree.ts`
- **Hierarchy Built in Code (`L162-L411`):**
  - **Root Priority Selector** (`root_selector`):
    - **Branch 1 (Priority 1):** `seq_heal` (Sequence)
      - `cond_health_low`: Is Health $< 35\%$?
      - `act_navigate_heal`: Steer to shrine, recover until $\geq 80\%$.
    - **Branch 2 (Priority 2):** `seq_exhaustion_recovery` (Sequence)
      - `cond_is_exhausted`: Is Agent Exhausted?
      - `act_catch_breath`: Disengage, maintain 180px standoff, 55% trot.

- **Branch 3 (Priority 3):** `seq_tactical_cover` (Sequence)
  - `cond_player_blocked`: Raycast check: direct line of sight occluded by crate?
  - `act_corner_flank`: Find nearest corner of blocking crate, path around it.
- **Branch 4 (Fallback):** `act_whisker_chase` (Action)
  - Multi-whisker predictive intercept pursuit.

```
// src/ai/behaviorTree.ts: L70-L86 (Selector Node Logic)
public tick(context: BTExecutionContext, activePath: string[]): BTNodeStatus {
  activePath.push(this.id);
  for (const child of this.children) {
    const status = child.tick(context, activePath);
    if (status === 'RUNNING' || status === 'SUCCESS') {
      this.lastStatus = status;
      return status; // First successful child wins!
    }
  }
  return 'FAILURE';
}
```

C. Decision Inputs Table

Input Name	Origin	Evaluation Method	Effect in Tree
`ctx.npc.health`	Entity state	`health <= maxHealth * 0.35`	Controls entry into `seq_heal`.
`ctx.crates`	Environment	`Math2D.hasLineOfSight(npc, player, crates)`	If false, enters `seq_tactical_cover` to corner-flank.
`sensor.hitDistance`	5 Ray Sensors	$U = (1 - d/L)^{1.8}$	Computes multi-feeler avoidance vector inside action leaves.

D. Worked Numerical Example

- **Inputs:** `health = 100%`, `isExhausted = false`, `hasLineOfSight = false` (crate blocks player).
- **Execution:**
  1. `root_selector` ticks Child 1 (`seq_heal`) → `cond_health_low` returns **FAILURE**. Sequence fails.
  2. `root_selector` ticks Child 2 (`seq_exhaustion_recovery`) → `cond_is_exhausted` returns **FAILURE**. Sequence fails.
  3. `root_selector` ticks Child 3 (`seq_tactical_cover`) → `cond_player_blocked` returns **SUCCESS**.
  4. Sequence proceeds to Child 2 (`act_corner_flank`). It calculates the 4 corners of the crate, picks the lowest cost corner, and returns **RUNNING**.
  5. Root Selector returns **RUNNING**. Active badge: `BT: Flank Cover`.

E. Professor Viva Questions & Answers

1. **Q: What is the fundamental operational difference between a Selector and a Sequence?**
   A: A **Selector** implements logical `OR` (ticks children until one returns `SUCCESS` or `RUNNING`). A **Sequence** implements logical `AND` (ticks children until one returns `FAILURE` or `RUNNING`).
2. **Q: What makes Behavior Trees superior to FSMs for complex games?**
   A: Modularity and sub-tree reusability. In an FSM, adding a new state requires wiring transitions to every existing state ($O(N^2)$ transitions). In a BT, you simply insert a new branch into a Selector without touching existing nodes.

4.3 Fuzzy Logic Controller

A. Core Idea & Analogy

- **Core Idea:** Replaces binary Boolean truth values (0 or 1) with continuous degrees of set membership ($\mu \in [0.0, 1.0]$). Multiple competing behaviors can be partially active at the same time and blended together smoothly.
- **Everyday Analogy:** Shower temperature controls. You don't jump between "100% Freezing" and "100% Boiling Scalding." You continuously blend hot and cold water to achieve "Comfortably Warm."

B. Implementation Details

- **Source File:** `src/ai/fuzzy.ts`
- **Primary Method:** `FuzzySteeringEngine.evaluate(npc, player, crates, healStations, nearestHealStation)`
- **Mathematical Membership Functions (`fuzzy.ts: L10-L24`):**
  - **Triangular (`trimf`):**

$$\mu(x; a, b, c) = \max\left(0, \min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right)\right)$$

  - **Trapezoidal (`trapmf`):**

$$\mu(x; a, b, c, d) = \max\left(0, \min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right)\right)$$

C. The 5 Fuzzy Rules Implemented in Code

```ts
// src/ai/fuzzy.ts: L67-L82
// Rule 1: IF Health is Low -> RETREAT & HEAL
const wRetreat = healthLow;

// Rule 2: IF Health is Med AND Distance is Near -> CAUTIOUS FLANK / ORBIT
const wFlank = Math.min(healthMed, distNear);

// Rule 3: IF Health is Med AND Distance is Med/Far -> BALANCED CHASE
const wBalancedChase = Math.min(healthMed, Math.max(distMed, distFar));

// Rule 4: IF Health is High AND Distance is Far -> AGGRESSIVE PURSUIT (Lead Target)
const wAggressiveChase = Math.min(healthHigh, distFar);

// Rule 5: IF Health is High AND Distance is Near -> RELENTLESS RAM
const wRelentless = Math.min(healthHigh, Math.max(distNear, distMed));
```

D. Defuzzification via Vector Weighted Centroid

Rather than defuzzifying to a single scalar steering angle, our engine performs **Vector Weighted Centroid Defuzzification**:

$$\vec{D}_{\text{blended}} = \frac{\sum_{i=1}^{5} (w_i \cdot k_i) \cdot \vec{V}_i}{\sum_{i=1}^{5} (w_i \cdot k_i)}$$

Where k_i are priority multipliers ($k_{\text{retreat}} = 1.6, k_{\text{flank}} = 1.1, k_{\text{balanced}} = 1.0, k_{\text{aggressive}} = 1.3, k_{\text{relentless}} = 1.4$).

Finally, obstacle repulsion is smoothly injected:

$$\vec{D}_{\text{final}} = \text{normalize}\left(\vec{D}_{\text{blended}} + (1.4 + 1.5 \cdot \text{Danger}_{\text{obstacle}}) \cdot \vec{F}_{\text{avoid}}\right)$$

E. Worked Numerical Example

- **Inputs:** `health = 35%`, `distToPlayer = 150px`, `minSensorDist = 120px` (clear).
- **1. Fuzzification:**
  - Health: `healthLow = trapmf(35, 0, 0, 25, 45) = (45 - 35) / (45 - 25) = 10 / 20 = 0.50`
  - Health: `healthMed = trimf(35, 30, 55, 75) = (35 - 30) / (55 - 30) = 5 / 25 = 0.20`
  - Health: `healthHigh = trapmf(35, 60, 80, 100, 100) = 0.00`
  - Distance: `distNear = trapmf(150, 0, 0, 90, 170) = (170 - 150) / (170 - 90) = 20 / 80 = 0.25`
  - Distance: `distMed = trimf(150, 110, 210, 310) = (150 - 110) / (210 - 110) = 40 / 100 = 0.40`
  - Distance: `distFar = 0.00`
- **2. Rule Inferences:**
  - `wRetreat = healthLow = 0.50`
  - `wFlank = min(healthMed, distNear) = min(0.20, 0.25) = 0.20`
  - `wBalancedChase = min(healthMed, max(0.40, 0)) = min(0.20, 0.40) = 0.20`
  - `wAggressiveChase = min(0, 0) = 0.00`
  - `wRelentless = min(0, max(0.25, 0.40)) = 0.00`
- **3. Defuzzification Blending:**

- Effective weights:
  - Retreat: $0.50 \times 1.6 = 0.80$
  - Flank: $0.20 \times 1.1 = 0.22$
  - Balanced: $0.20 \times 1.0 = 0.20$
  - Total Weight = $0.80 + 0.22 + 0.20 = 1.22$
- The steering vector is 65% **Retreat,** 18% **Flank, and** 17% **Chase**!
- **Behavior Result:** The NPC begins retreating toward the health shrine while smoothly angling away from the player without any snapping or jitter.

F. Professor Viva Questions & Answers

1. **Q: Is defuzzification in your project Sugeno or Mamdani?**
   A: It is a continuous **Mamdani-style inference** model using vector-weighted centroid blending. Each rule's firing strength scales a directional behavior vector, and their weighted centroid produces the final desired velocity.
2. **Q: Why does the profiler show Fuzzy Logic consuming ~60µs (the highest of all algorithms)?**
   A: Fuzzy logic evaluates continuous mathematical functions across all membership sets, evaluates multiple min/max logic operations, constructs 5 candidate geometric vectors, and computes vector centroid normalization on *every single frame*. Unlike FSM which evaluates 1 condition and exits, Fuzzy logic must evaluate *everything* continuously.

4.4 Goal-Oriented Action Planning (GOAP)

A. Core Idea & Analogy

- **Core Idea:** Separates the *goal* from the *actions*. Given a current world state and a desired goal state, an A^* graph search backwards- or forwards-chains through an action dictionary to discover the lowest-cost sequence of steps.
- **Everyday Analogy:** GPS navigation. You specify your destination (Goal). The GPS searches through road segments (Actions) based on traffic costs and gives you a turn-by-turn plan.

B. Implementation Details

- **Source File:** `src/ai/goap.ts`
- **World State Atoms (`goap.ts: L11-L13`):**
  - `isHealthy`, `isCriticallyInjured`, `atHealingStation`, `hasLineOfSight`, `inCombatRange`, `inMeleeRange`.
- **Goals Implemented:**
  1. `Survive & Regenerate`: Priority 95 if $\text{HP} \leq 35$. Desired: `{ isHealthy: true }`.
  2. `Flank Obscured Player`: Priority 70 if $\text{LOS} = \text{false}$. Desired: `{ hasLineOfSight: true, inCombatRange: true }`.
  3. `Strike Player`: Priority 50. Desired: `{ inMeleeRange: true }`.
- **Actions in Pool (`goap.ts: L64-L175`):**
  - `Navigate to Healing Shrine` $\rightarrow$ Effects: `{ atHealingStation: true }`
  - `Regenerate HP in Shrine` $\rightarrow$ Preconditions: `{ atHealingStation: true }`, Effects: `{ isHealthy: true }`

- `Corner Flank Maneuver` → Preconditions: `{ hasLineOfSight: false }`, Effects: `{ hasLineOfSight: true, inCombatRange: true }`
- `Predictive Intercept Gap` → Preconditions: `{ hasLineOfSight: true }`, Effects: `{ inCombatRange: true }`
- `Melee Pounce Attack` → Preconditions: `{ inCombatRange: true, hasLineOfSight: true }`, Effects: `{ inMeleeRange: true }`

```
// src/ai/goap.ts: L193-L232 (A* Forward Plan Search)
while (openSet.length > 0) {
  openSet.sort((a, b) => a.fCost - b.fCost);
  const current = openSet.shift()!;
  if (this.satisfies(current.state, goalState)) {
    return reconstructPlan(current);
  }
  // Expand actions whose preconditions are satisfied by current.state
  for (const action of this.actions) {
    if (this.satisfies(current.state, action.preconditions)) {
      const nextState = this.applyEffects(current.state, action.effects);
      // gCost = current.gCost + action.getCost(...)
      // hCost = unsatisfied goal variables count
      openSet.push({ state: nextState, action, parent: current, fCost: gCost + hCost });
    }
  }
}
```

C. Worked Numerical Example

- **Initial World State:** `{ isHealthy: false, atHealingStation: false }` ($\text{HP} = 20$).
- **Goal Selected:** `Survive & Regenerate` (Priority 95, Desired: `{ isHealthy: true }`).
- **A* Search Tree:**
  - *Node 0:* `{ isHealthy: false, atHealingStation: false }` (g=0, h=1, f=1).
  - *Expand Actions:*
    - `Navigate to Healing Shrine`: Preconditions `{}` met. Next state: `{ atHealingStation: true }`. Cost = $d/120 = 2.5$. $g = 2.5, h = 1, f = 3.5$.
    - `Melee Pounce Attack`: Preconditions `{ inCombatRange: true }` NOT met. Skipped.
  - *Node 1 Expanded:* `{ atHealingStation: true }`.
    - `Regenerate HP in Shrine`: Preconditions `{ atHealingStation: true }` met! Next state: `{ isHealthy: true }`. Cost = 1.0. $g = 3.5, h = 0, f = 3.5$.
  - *Goal Satisfied!*
- **Generated Plan:** `[Navigate to Healing Shrine] -> [Regenerate HP in Shrine]`.

D. Professor Viva Questions & Answers

1. **Q: How does GOAP handle a dynamic environment if an obstacle suddenly appears?**
   A: GOAP validates action completion on every tick. If the current action becomes invalid or if world state changes (e.g. line of sight is lost), GOAP clears the active plan and re-runs A^* search from the new initial state.
2. **Q: Why does GOAP cost ~24µs in our profiler compared to 2.8µs for FSM?**
   A: FSM evaluates static if-else branches. GOAP clones state dictionaries, performs sorting on a priority queue, and searches a combinatorial state-space graph on every replanning tick.

4.5 Utility AI (Response Curves & Inertia Hysteresis)

A. Core Idea & Analogy

- **Core Idea:** Every candidate action evaluates a continuous mathematical function representing its "usefulness" (utility $U \in [0.0, 1.0]$) given the current situation. The action with the highest utility wins.
- **Everyday Analogy:** Deciding what to eat. You score Pizza, Salad, and Burger based on Hunger, Cost, and Healthiness. You choose whatever has the highest total score.

B. Implementation Details

- **Source File:** `src/ai/utility.ts`
- **Primary Method:** `UtilityAIEngine.evaluate(...)`
- **Response Curves Implemented (`utility.ts: L41-L180`):**
  1. **Emergency Heal Retreat (Exponential Curve):**

$$U_{\text{heal}} = \min\left(1.0, (1 - \text{hpRatio})^{2.2} \times 1.4 \times \text{distFactor}\right) \times 1.25$$

  2. **Tactical Corner Flank (Logistic Sigmoid Curve):**

$$U_{\text{flank}} = 0.82 \times \frac{1}{1 + e^{-6(d_{\text{norm}} - 0.3)}} \times \text{hpRatio}$$

  3. **Predictive Intercept (Linear Response Curve):**

$$U_{\text{intercept}} = (0.45 + 0.35 \cdot d_{\text{norm}} + \text{motionBonus}) \times \text{hpRatio}$$

  4. **High-Aggression Melee Pounce (Quadratic Proximity Curve):**

$$U_{\text{pounce}} = \left(1 - \frac{d}{180}\right)^2 \times 0.95 \times \text{hpConfidence} \times \text{staminaFactor} \times 1.15$$

- **Inertia Hysteresis Bonus (`utility.ts: L190-L218`):** To prevent rapid, unappealing flip-flopping between two actions with close scores (e.g. $U_1 = 0.71, U_2 = 0.72$), the currently executing action receives a $+0.12$ (12%) **inertia bonus**:

$$U_{\text{final}} = U_{\text{raw}} + (\text{isCurrentAction} ? 0.12 : 0)$$

Part 5: Direct Comparative Analysis Matrix

Evaluation Dimension	Simple FSM	Behavior Tree	Fuzzy Logic	GOAP Planner	Utility AI
Decision Representation	Discrete states (`FLEE`, `CHASE`)	Hierarchical composite tree	Continuous rule weights & membership	Graph of action preconditions & effects	Mathematical response curve equations

Evaluation Dimension	Simple FSM	Behavior Tree	Fuzzy Logic	GOAP Planner	Utility AI
Action Selection Mechanism	Hard `if-else` condition ladder	Left-to-right Selector/Sequence traversal	Mamdani min-max rule evaluation	A^* heuristic forward state search	$\text{argmax}(\text{Utility}_i - \text{Inertia})$
Transition Quality	Rigid; snaps abruptly across thresholds	Discrete step transitions	**Completely smooth continuous blending**	Step-by-step sequential execution	Continuous, inertia-damped transitions
Handling Multiple Pressures	Fails; only one state can be active	Handled by strict node priority	**Exemplary; simultaneously flees & flanks**	Formulates multi-step contingency plans	Balances multiple normalized considerations
Computational Overhead	$2.8\mu s$ **(** $1.0\times$**)**	$34.4\mu s$ $(12.3\times)$	$60.8\mu s$ $(21.7\times$)	$24.6\mu s$ $(8.8\times)$	$27.9\mu s$ $(10.0\times)$
Authoring Complexity	Simple initially; scales horribly ($O(N^2)$)	Highly modular, easy to expand	Requires tuning membership curve bounds	Requires defining precise preconditions/effects	Requires mathematical curve tuning
Best Suited For	Simple ambient critters	Structured tactical combat	Smooth driving, vehicle steering, organic NPCs	Long-term strategy, puzzles, stealth	Open-world emerge decision making

Part 6: Masterclass on Fuzzy Logic for NPC Steering

6.1 Why Fuzzy Logic for Game Steering?

In steering AI, an agent must frequently negotiate contradictory requirements:

- "The enemy is wounded, so I should attack."
- "My own health is moderate, so I should be cautious."
- "An obstacle is near, so I must turn."

In an FSM or Behavior Tree, one of these rules must win completely, forcing the agent to either ignore the enemy or ignore its own health. **Fuzzy logic is the only architecture where all three considerations contribute proportionally to the steering heading.**

6.2 The Three Fundamental Steps of Our Engine

```
[ Crisp Inputs: HP=35%, Dist=150px ]
                |
                v
   STEP 1: FUZZIFICATION (trimf, trapmf)
   ==> healthLow=0.50, healthMed=0.20, distNear=0.25, distMed=0.40
                |
                v
   STEP 2: MAMDANI INFERENCE (Min-Max)
   ==> Rule 1 (Retreat)=0.50, Rule 2 (Flank)=0.20, Rule 3 (Chase)=0.20
                |
                v
   STEP 3: DEFUZZIFICATION (Vector Weighted Centroid)
   ==> Blended Vector = (0.65 * V_Heal) + (0.18 * V_Flank) + (0.17 * V_Chase)
                |
                v
[ Output: Smooth, organic trajectory curving away from player toward shrine ]
```

Part 7: Experimental Methodology, Scenarios & Scientific Fairness

7.1 How Controlled Experiments Are Executed

Automated experiments are orchestrated by `src/game/ExperimentManager.ts` and `src/game/GameEngine.ts: L1177-L1236`.

- **Deterministic Seeds:** Every trial index uses a strictly deterministic pseudo-random seed:

$$\text{Seed} = 100000 + \text{trialIdx} \cdot 37$$

- **The Benchmark Opponent:** During trials, the player is not human-controlled. It is driven by `updateAutomatedPlayer()`, executing deterministic kiting, wall-repulsion, and automated pulse shockwave defense at exact distance thresholds ($d < 135\text{px}$).
- **Zero Initial Confounding Bias:** In every trial, both NPC and opponent spawn at the exact same coordinates, with identical hitpoints and time limits.

7.2 The 5 Standardized Benchmark Scenarios

Scenario ID	Name	Time Limit	Obstacles & Shrines	Purpose & What It Measures
`open_arena`	**Scenario 1: Open Arena**	30s	None	Measures raw pursuit speed, line-of-sight interception, and direct combat damage output.
`obstacle_arena`	**Scenario 2: Obstacle Arena**	35s	6 symmetrical pillars	Tests continuous 5-ray whisker obstacle avoidance, steering jitter, and navigation around static barriers.
`corridors`	**Scenario 3: Corridors & Choke Points**	40s	5 choke barriers	Evaluates spatial adaptation, cornering resilience, and positioning under tight geometric bounds.

Scenario ID	Name	Time Limit	Obstacles & Shrines	Purpose & What It Measures
`low_health`	**Scenario 4: Low-Health Survival**	35s	3 bunkers, 1 NW shrine	NPC starts at **critical 25% HP**. Measures self-preservation triggering, retreat execution, and pathfinding under crisis.
`healing_shrines`	**Scenario 5: Healing Shrines**	40s	2 crates, 2 healing shrines	Both entities start at **50% HP**. Compares dynamic balancing between attacking wounded player vs retreating to recover.

Part 8: Performance Metrics, Statistical Analysis & CSV Generation

8.1 Evaluated Metrics & Definitions

Every trial records eight core metrics in `src/game/TrialMetricsCollector.ts`:

1. **Survival Time (T_{surv}):** Seconds elapsed before NPC death or trial timeout ($[0, T_{\text{limit}}]$).
2. **Damage Dealt (D_{dealt}):** Total cumulative HP stripped from the opponent via melee contact ($22\ \text{HP}/\text{sec}$).
3. **Damage Received (D_{recv}):** Cumulative damage absorbed from the player's EMP shockwave pulses ($35\ \text{HP}/\text{blast}$).
4. **Attacks Count:** Number of discrete, debounced offensive melee contacts (debounced with 0.4s cooldown).
5. **Retreats Count:** Number of discrete state transitions into a healing/retreat state.
6. **Distance Travelled:** Cumulative Euclidean path length: $\sum \sqrt{\Delta x^2 + \Delta y^2}$.
7. **Win Rate (%):** Percentage of trials where the NPC destroyed the opponent or had higher HP at timeout.
8. **Final Health:** Remaining hitpoints ($[0, 100]$) at trial termination.

8.2 Statistical Aggregation Formulas

For an evaluation run of N trials, the system computes:

- **Sample Mean ($\bar{x}$):**

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i$$

- **Sample Standard Deviation (s):**

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N} (x_i - \bar{x})^2}$$

All data exports directly to RFC-4180 compliant CSV format via `exportTrialsToCSV()`, allowing direct import into Python (Pandas/Matplotlib) or Excel.

Part 9: End-to-End Execution Flowchart & Code Traces

Trace: From Browser Click to Movement Execution

```
1. Browser fires tick: requestAnimationFrame(render) in GameCanvas.tsx
2. GameEngine.update(dt) is called (GameEngine.ts: L457)
3. updateSensors(npc) casts 5 rays against all crates (GameEngine.ts: L1010)
4. GameEngine calls FuzzySteeringEngine.evaluate(...) (GameEngine.ts: L644)
   a. Calculates trimf/trapmf memberships for distance, health, obstacles
   b. Fires 5 fuzzy rules (Mamdani Min-Max)
   c. Performs vector-weighted centroid defuzzification
   d. Injects whisker obstacle avoidance force
   e. Clamps steering force: F = clampLength(V_desired - V_current, F_max)
5. GameEngine adds flocking separation force from nearby NPCs (GameEngine.ts: L734)
6. GameEngine drains/recharges aggression stamina (GameEngine.ts: L754)
7. Physics integration: V += F_steering * dt; P += V * dt (GameEngine.ts: L795)
8. Resolve AABB crate collisions (Math2D.resolveBoxCollision)
9. Resolve healing shrine health ticks and combat contact damage
10. If in experiment mode, update TrialMetricsTracker.onStep()
11. GameCanvas.drawScene() renders avatar, whiskers, vectors, and state badge
```

Part 10: Presentation & Viva Defense Preparation (25+ Questions & Answers)

10.1 The 2-Minute Project Pitch (Spoken Script)

> "Good morning, respected professors. My project investigates: *'How does fuzzy-logic decision making affect NPC behavior compared with alternative AI decision architectures?'* In video games and autonomous robotics, NPCs often suffer from rigid binary thresholds—for example, abruptly snapping between aggressive pursuit and fleeing when crossing an exact 30% health boundary. To investigate this, I developed a high-performance simulation platform in TypeScript and HTML5 Canvas. We implemented five complete AI decision architectures controlling identical vehicle-steering NPCs: a Finite State Machine, Behavior Trees, Goal-Oriented Action Planning (GOAP), Utility AI, and a Fuzzy Logic Controller. All five architectures operate under strictly identical physical conditions: Reynolds steering forces, a 5-feeler raycast whisker sensory array, and an aggression stamina system. Furthermore, I built an automated, deterministic benchmark system featuring five controlled scenarios —such as Open Arenas, Obstacle Pillar Grids, and Corridors. An automated benchmark opponent ensures zero human bias. Our experimental metrics prove that while Simple FSM is computationally fastest at 2.8 microseconds, it suffers from severe trajectory jitter. Fuzzy Logic, while requiring 60 microseconds due to continuous Mamdani inference, produces mathematically superior, smooth transitions by simultaneously blending offensive pursuit, tactical flanking, and emergency retreat."

10.2 Top 25 Viva Questions & Defensible Answers

Architectural & Algorithmic Questions

1. **Q: Why compare Fuzzy Logic with four other architectures instead of just an FSM?**
   A: Comparing only with FSM would be a strawman comparison. Behavior Trees, GOAP, and Utility AI represent modern game industry standards. Comparing across all five proves where Fuzzy Logic stands against both state-based and planning-based paradigms.
2. **Q: How does Utility AI differ fundamentally from Fuzzy Logic?**
   A: Both use continuous curves. However, **Utility AI evaluates competing actions and selects one winner** (argmax). **Fuzzy Logic blends multiple active rules simultaneously** using vector defuzzification.
3. **Q: What is the purpose of Reynolds Steering in your simulation?**
   A: It separates decision-making from physics. The AI computes a desired velocity vector $\vec{V}_{\text{desired}}$, and Reynolds steering applies a bounded force $\vec{F} = \vec{V}_{\text{desired}} - \vec{V}_{\text{actual}}$ to create natural acceleration, turning momentum, and realistic cornering.
4. **Q: Explain how whisker raycasting works in your code.**
   A: Each NPC casts five ray sensors at angles $0°$, $\pm 24°$ (± 0.42 rad), and $\pm 49°$ (± 0.85 rad). Rays intersect crate bounding boxes via `rayIntersectAABB()`. The closer an obstacle is, the stronger the repulsive normal force applied: $F_{\text{avoid}} \propto (1 - d/L)^2$.
5. **Q: What is the role of Aggression Stamina?**
   A: In high-aggression pursuit, stamina drains over 4 seconds. When depleted, `isExhausted` becomes true, dropping speed to 55% for 5 seconds. This creates tactical pacing, allowing players to escape and preventing endless chasing.

Code & Implementation Questions

6. **Q: Where is the main simulation loop located?**
   A: In `src/game/GameEngine.ts` inside the `update(rawDt: number)` method (lines 457–540).
7. **Q: How is deterministic seed generation achieved in automated trials?**
   A: In `src/game/ExperimentManager.ts: L100`, each trial uses `seed = 100000 + trialIdx * 37`.
8. **Q: How do you prevent NPCs from clustering on top of each other?**
   A: We use a two-phase system: a proactive inverse-square flocking separation force (`GameEngine.ts: L734-L752`) and an explicit two-pass rigid-body non-penetration relaxation solver (`GameEngine.ts: L898-L945`).
9. **Q: What defuzzification algorithm is used in `fuzzy.ts`?**
   A: Vector Weighted Centroid defuzzification (`fuzzy.ts: L134-L165`). It computes the normalized vector sum of each rule's action vector scaled by its firing strength.
10. **Q: Why does `PerformanceGraph.tsx` use an Exponential Moving Average (EMA)?**
    A: CPU scheduling and browser garbage collection cause single-frame microsecond spikes. EMA with $\alpha = 0.12$ filters noise to display smooth, statistically reliable computational costs.

Experimental Methodology & Fairness Questions

11. **Q: How do you guarantee that experiments are fair across all 5 AI models?**
    A: All 5 models face the exact same automated opponent (`updateAutomatedPlayer`), in the exact same scenario maps, with identical starting HP, coordinates, and deterministic seeds.
12. **Q: What is the automated opponent logic?**
    A: It kites away when $d < 190\text{px}$, advances when $d > 340\text{px}$, strafes perpendicularly between 190–

340px, avoids crates, and fires an EMP shockwave when an NPC enters 135px.

13. **Q: What are the primary threats to validity in your experimental setup?**
    A: The automated opponent uses a fixed engagement distance (180px). If an AI architecture is tuned specifically for that distance, it may perform better. To mitigate this, we test across five diverse scenarios with varying geometry and health conditions.
14. **Q: Why is damage dealt alone an insufficient metric for AI quality?**
    A: An AI could blindly charge forward and deal high damage before immediately dying. Survival time, retreat count, and health efficiency must be examined together.
15. **Q: What does the sample standard deviation (s) tell us in the metrics table?**
    A: It measures behavioral consistency. A low standard deviation means the AI behaves deterministically and reliably; a high standard deviation indicates sensitivity to initial positions or spatial trapping.

Part 11: 7-Day Structured Study Plan & Ultimate Exam Cheat Sheet

11.1 The 7-Day Viva Preparation Plan

```
+-------------------------------------------------------------------------------------+
|                                7-DAY STUDY CURRICULUM                               |
+-------+---------------------------------+-------------------------------------------+
| Day   | Focus Area                      | Files to Read & Self-Test Objective       |
+-------+---------------------------------+-------------------------------------------+
| Day 1 | Project Overview & Profiler     | Inspect PerformanceProfiler.ts & UI image. |
|       |                                 | Explain all metrics in the profiler panel. |
| Day 2 | Simulation Loop & Physics       | Read GameEngine.ts update() & Math2D.ts.   |
|       |                                 | Trace a full tick from sensing to physics. |
| Day 3 | Simple FSM & Behavior Trees     | Read simpleLogic.ts & behaviorTree.ts.     |
|       |                                 | Explain Selector vs Sequence & FSM jitter. |
| Day 4 | Fuzzy Logic Masterclass         | Read fuzzy.ts. Calculate worked numerical  |
|       |                                 | example by hand using trimf/trapmf.        |
| Day 5 | GOAP & Utility AI               | Read goap.ts & utility.ts. Explain A* plan |
|       |                                 | search & response curve inertia bonuses.   |
| Day 6 | Benchmarking & Metrics          | Read ExperimentManager.ts & Scenarios.     |
|       |                                 | Defend fairness & statistical aggregation. |
| Day 7 | Mock Viva & Pitch Practice      | Practice 2-minute pitch & answer the 25    |
|       |                                 | viva questions out loud without notes.     |
+-------+---------------------------------+-------------------------------------------+
```

11.2 Ultimate Viva Cheat Sheet

- **Research Question:** *"How does fuzzy-logic decision making affect NPC behavior compared with alternative AI decision architectures?"*
- **The Core Trade-off:**
  - **FSM:** Fastest ($2.8\mu\text{s}$), simplest, but rigid, jittery at boundaries.
  - **Behavior Tree:** Highly modular ($34.4\mu\text{s}$), priority-ordered, standard for complex game logic.
  - **Fuzzy Logic:** Most fluid & organic ($60.8\mu\text{s}$), blends multiple goals, zero threshold jitter.
  - **GOAP:** Strategic planning ($24.6\mu\text{s}$), decouples actions from goals, A^* state-space search.
  - **Utility AI:** Highly nuanced ($27.9\mu\text{s}$), continuous scoring curves, inertia damping prevents thrashing.

- **Steering Formula:** $\vec{F}_{\text{steering}} = \text{clampLength}(\vec{V}_{\text{desired}} - \vec{V}_{\text{current}}, F_{\max})$.
- **Sensory Whiskers:** 5 ray sensors ($0^\circ, \pm 24^\circ, \pm 49^\circ$) repelling proportionally to $(1 - d/L)^2$.
- **Automated Benchmark Opponent:** Deterministic kiting (180px standoff) + EMP pulse defense (135px, 35 damage).